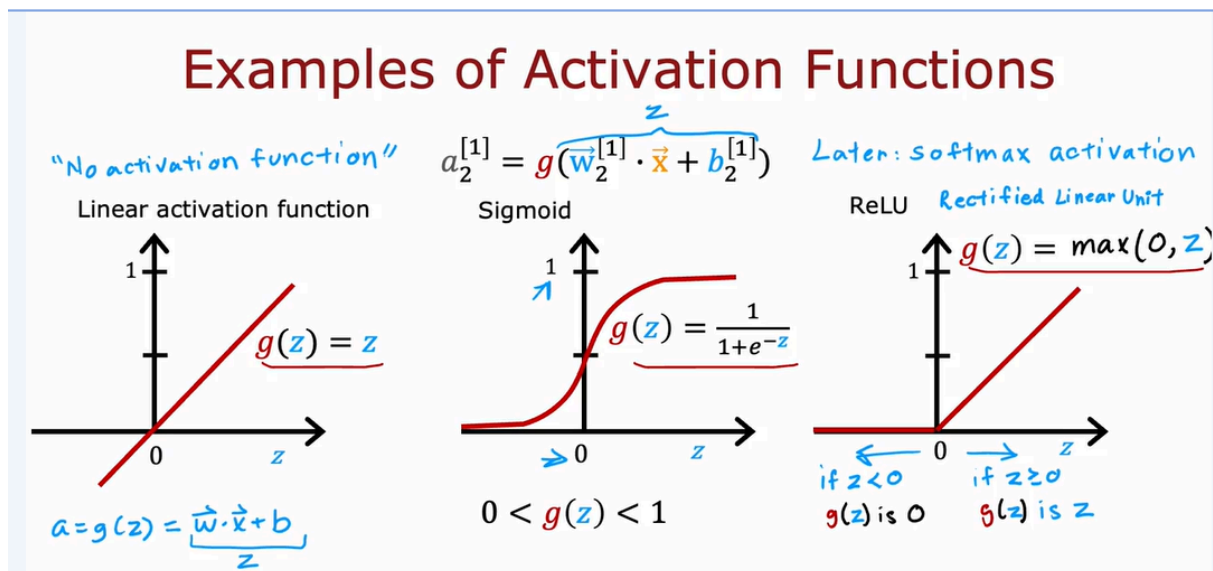


Activation Functions

Alternatives to the sigmoid activation

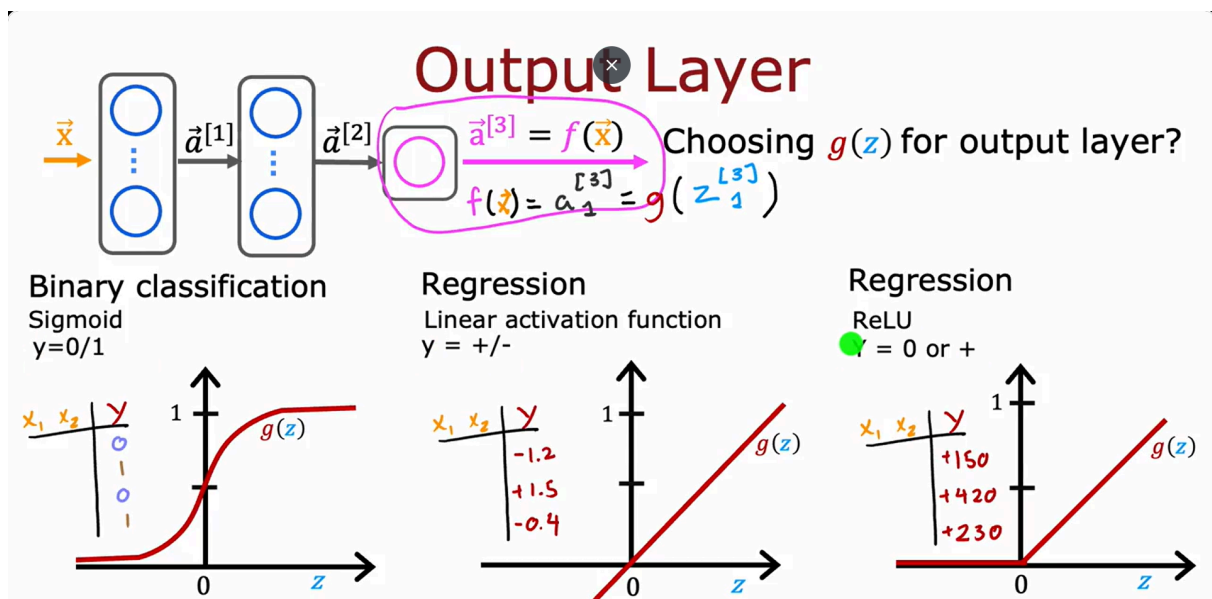


There are a lot of Activation Functions for us to use.

Sometimes if you use the linear activation function, people will say we're not using any activation function because if a is $g(z)$ where $g(z) = z$. And so it's as if there was no g in there at all. So when you are using this linear activation function $g(z)$ sometimes people say, well, we're not using any activation function.

Although in this class, I will refer to using the linear activation function rather than no activation function. But if you hear someone else use that terminology, that's what they mean. It just refers to the **linear activation function**. And these three are probably by far the most commonly used activation functions in neural networks.

Choosing activation functions



Sigmoid activation function

Specifically, if you are working on a classification problem where y is either zero or one, so a binary classification problem, then the sigmoid activation function will almost always be the most natural choice, because then the neural network learns to predict the probability that y is equal to one, just like we had for logistic regression. My recommendation is, if you're working on a binary classification problem, use sigmoid at the output layer.

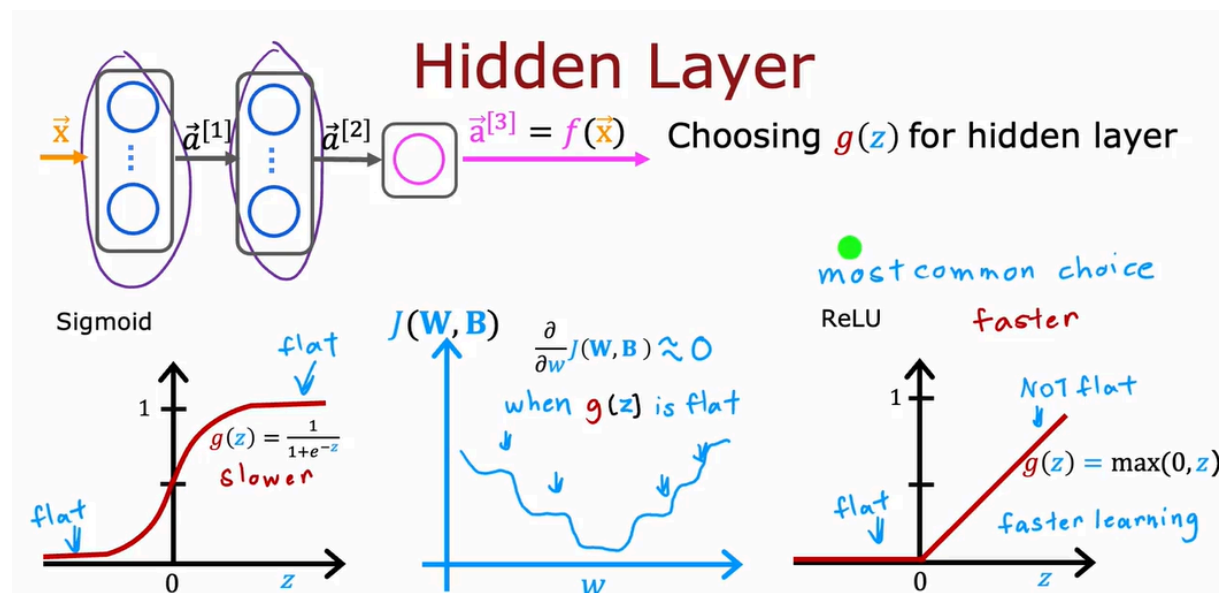
Linear activation function

Alternatively, if you're solving a regression problem, then you might choose a different activation function. For example, if you are trying to predict how tomorrow's stock price will change compared to today's stock price. Well, it can go up or down, and so in this case y would be a number that can be either positive(+) or negative(-), and in that case I would recommend you use the linear activation function. Why is that? Well, that's because then the outputs of your neural network, $f(x)$, which is equal to $a^{[3]}$ in the example above, would be $g(z^{[3]})$ and with the linear activation function, $g(z)$ can take on either positive or negative values. So y can be positive or negative, use a linear activation function.

Rectified Linear Unit (ReLU)

Finally, if y can only take on non-negative values, such as if you're predicting the price of a house, that can never be negative, then the most natural choice will be the ReLU activation function because as you see here, this activation

function only takes on non-negative values, either zero or positive values. In choosing the activation function to use for your output layer, usually depending on what is the label y you're trying to predict, there'll be one fairly natural choice.

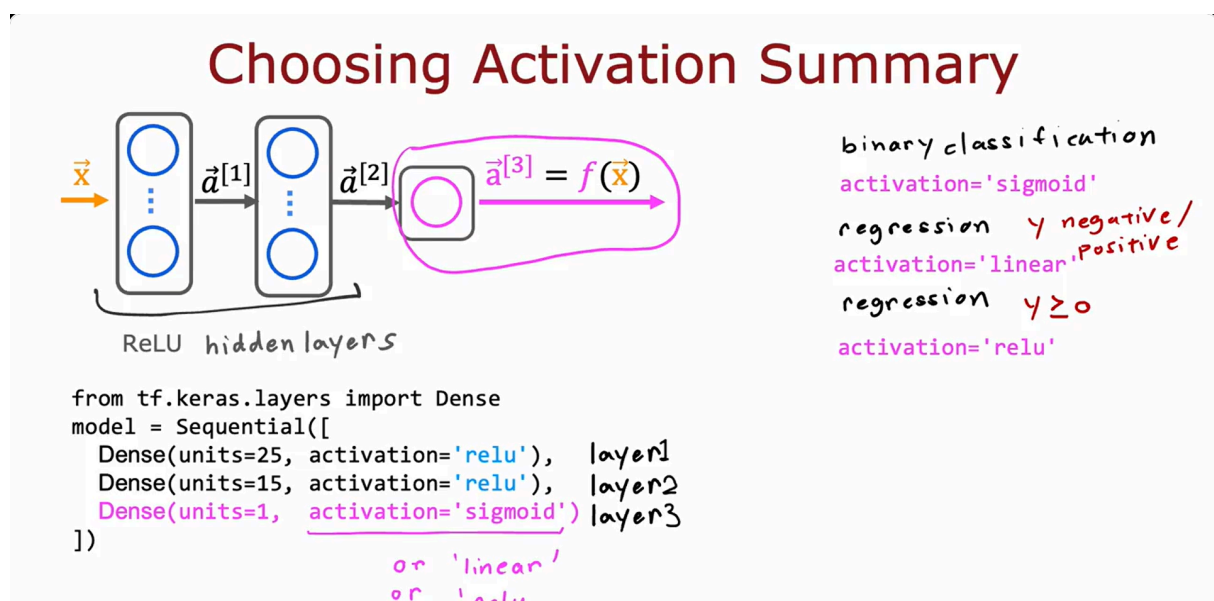


Even though we had initially described neural networks using the sigmoid activation function, and in fact, in the early history of the development of neural networks, people use sigmoid activation functions in many places, the field has evolved to use ReLU much more often and sigmoids hardly ever.

Well, the one exception that you do use a sigmoid activation function in the output layer if you have a binary classification problem. So why is that? Well, there are a few reasons.

- First, if you compare the ReLU and the sigmoid activation functions, the ReLU is a bit faster to compute because it just requires computing $\max(0, z)$, whereas the sigmoid requires taking an exponentiation and then an inverse and so on, and so it's a little bit less efficient. But the second reason which turns out to be even more important is that the ReLU function goes flat only in one part of the graph; here on the left is completely flat, whereas the sigmoid activation function, it goes flat in two places. It goes flat to the left of the graph and it goes flat to the right of the graph. If you're using gradient descent to train a neural network, then when you have a function that is flat in a lot of places, gradient descents would be really slow. I know that gradient descent optimizes the cost function $J(W, B)$ rather than optimizes the activation function, but the activation function is a piece of

what goes into computing, and that results in more places in the cost function $J(W, B)$ that are flats as well and with a small gradient and it slows down learning. I know that that was just an intuitive explanation, but researchers have found that using the ReLU activation function can cause your neural network to learn a bit faster as well, which is why for most practitioners if you're trying to decide what activation functions to use with hidden layer, the ReLU activation function has become now by far the most common choice.



→ The hidden layers should use ReLU activation function

→ The output layer can use any activation functions, it depends on the expected output

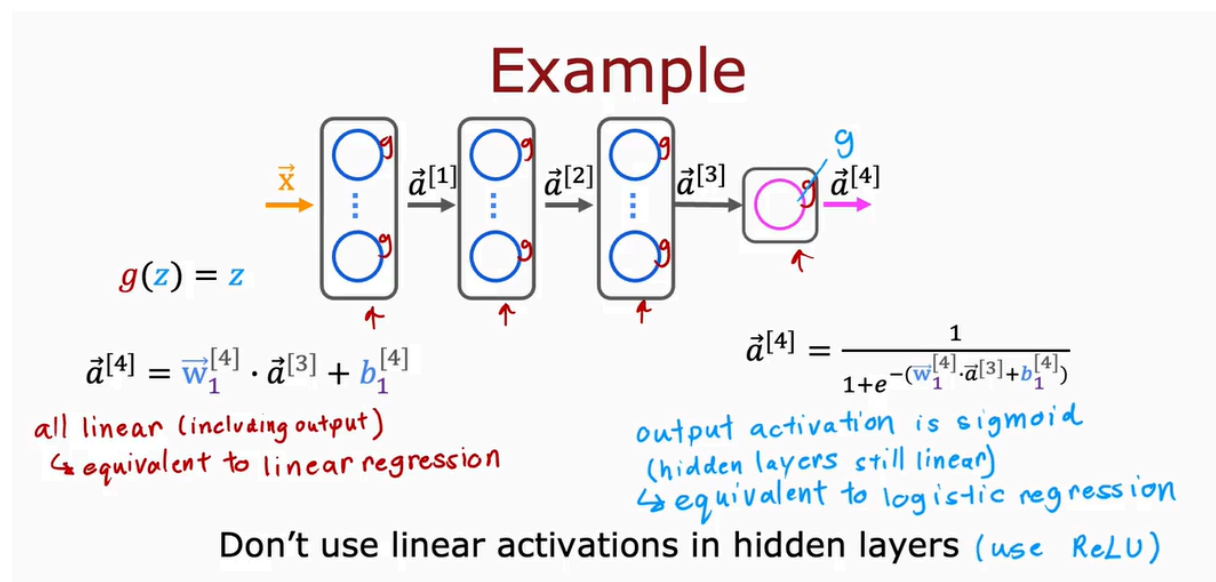
By the way, if you look at the research literature, you sometimes hear of authors using even other activation functions, such as the tanh activation function or the LeakyReLU activation function or the swish activation function. Every few years, researchers sometimes come up with another interesting activation function, and sometimes they do work a little bit better. For example, I've used the LeakyReLU activation function a few times in my work, and sometimes it works a little bit better than the ReLU activation function you've learned about in this video. But I think for the most part, and for the vast majority of applications what you learned about in this video would be good enough. Of course, if you want to learn more about other activation functions, feel free to look on the Internet, and there are just a small handful of cases where these other activation functions could be even more powerful as well.

With that, I hope you also enjoy practicing these ideas, these activation functions in the optional labs and in the practice labs. But this raises yet another question

More information

The tanh (hyperbolic tangent) activation function is a popular choice in neural networks, especially for hidden layers. It maps input values to a range between -1 and 1, providing a zero-centered output that can help improve training speed and convergence. Its formula is $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$.

Why do we need activation functions?



if you had a neural network with multiple layers like this and say you were to use a linear activation function for all of the hidden layers and also use a linear activation function for the output layer, then it turns out this model will compute an output that is completely equivalent to linear regression. The output $\vec{a}^{[4]}$ can be expressed as a linear function of the input features x plus b .

Or alternatively, if we were to still use a linear activation function for all the hidden layers, for these three hidden layers here, but we were to use a logistic activation function for the output layer, then it turns out you can show that this model becomes equivalent to logistic regression, and $\vec{a}^{[4]}$, in this case, can be expressed as $\vec{a}^{[4]} = \frac{1}{1 + e^{-(\vec{w}_1^{[4]} \cdot \vec{a}^{[3]} + b_1^{[4]})}}$ for some values of w and b . So this big neural network doesn't do anything that you can't also do with logistic

regression. That's why a common rule of thumb is don't use the linear activation function in the hidden layers of the neural network. In fact, I recommend typically using the ReLU activation function should do just fine.